JAVA CHAPTER 9 — ANNOTATIONS: METADATA, RETENTION & COMMON USES

Declarative metadata • Guide tools and frameworks • Reduce boilerplate

Smarter Code with Annotations

1 WHY IT MATTERS

Small Code. Big Impact.

Add Metadata
Annotations add metadata without changing business logic.

Used Widely
Heavily used in frameworks, validation, serialization, testing and dependency injection.

Improve Development
They improve readability and automation.

2 MENTAL MODEL

Annotations describe intent, not behavior.

Source code element → Annotation attached → Compiler / tools / framework reads it → Behavior or validation changes

- Class, method, field, etc.
- e.g. @Override or custom
- During compilation or at runtime
- Generates warnings, applies configuration, enables features, etc.

- **Annotations describe intent.**
- **They usually do not execute code by themselves.**

3 BUILT-IN ANNOTATIONS

Useful out of the box.

Annotation	Meaning
@Override	Indicates a method overrides a superclass method (compiler checks).
@Deprecated	Marks an element as deprecated (shows a warning when used).
@SuppressWarnings	Suppresses specific compiler warnings.
@FunctionalInterface	Marks that an interface is a functional interface (one abstract method).
@SafeVarargs	Asserts that a method with varargs does not perform unsafe operations.

4 CUSTOM ANNOTATION SYNTAX

Define your own annotations.

```java
import java.lang.annotation.*;

@Target({ElementType.METHOD, ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
public @interface Audit {
   String value() default "";  // required element
   int level() default 1;      // element with default
}
```

Usage:

```java
@Audit(value = "login",
       levell = 2)
public void doLogin() {
    ....
}
```

- Annotation elements can have **default values.**
- Use meta-annotations to control where and how it can be used.

5 META-ANNOTATIONS

Define behavior for annotations.

Meta-Annotation	Purpose
@Target	Specifies where the annotation can be used (e.g. type, method, field).
@Retention	Specifies how long the annotation is retained (SOURCE, CLASS, RUNTIME).
@Documented	Includes the annotation in Javadoc documentation.
@Inherited	Allows annotations on a class to be inherited by subclasses.
@Repeatable	Allows multiple instances of the same annotation on a single element.

6 RETENTION LEVELS

Controls when annotations are available.

Retention Policy	Where It's Available	Key Points
SOURCE	Visible in source code only (ignored by compiler).	Not stored in class file. Not available at runtime.
CLASS	Stored in the .class file (by compiler).	Available to tools during build-time. Not available through reflection at runtime.
RUNTIME	Available through reflection at runtime.	Required if frameworks or code need to read the annotation at runtime.

To read annotations using reflection at runtime, the annotation must have `@Retention(RetentionPolicy.RUNTIME)`.

7 COMMON REAL-WORLD USES

Power many Java frameworks.

Validation	e.g. @NotNull, @Size (Bean Validation).
Dependency Injection	e.g. @Inject, @Autowired (Spring).
Mapping / Serialization	e.g. @JsonProperty, @Entity (Jackson, JPA).
Test Methods	e.g. @Test, @BeforeEach (JUnit).
Configuration	e.g. @Configuration, @Bean (Spring).

8 COMMON MISTAKES

Avoid common pitfalls.

- Assuming annotations run by themselves.
- Forgetting the retention policy.
- Using the wrong target (e.g. method vs. class).
- Overusing @SuppressWarnings.
- Mixing compile-time and runtime use cases.

9 QUICK RULES / AT A GLANCE

Key takeaways to remember.

1. Use annotations to add metadata, not business logic.
2. Choose the correct retention policy.
3. Set the appropriate target.
4. Prefer built-in annotations when available.
5. Use runtime retention only when you need reflection.

10 JAVA EXAMPLE

Reflection can read annotations.

Annotation definition and usage:

```java
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface LogExecution {
    String value() default "";
}

public class Service {
    @LogExecution("calculate")
    public int calculate(int a, int b) {
        return a + b;
    }
}
```

Result (using reflection):

```java
Method m = Service.class
    .getMethod("calculate",
        int.class, int.class);
LogExecution log =
    m.getAnnotation(
        LogExecution.class);
System.out.println(
    log.value()); // "calculate"
```